

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 7. String and its Methods

Lecture: 69. Introduction to String

Section 1: Lecture Summary

This lecture introduces the concept of strings in Python. It begins by defining a string as a collection of characters enclosed in quotes and explaining that strings may contain alphabets, numbers, special characters, and spaces. The lecture then covers string representation in Python memory as a sequence (similar to arrays), including the concept of positive and negative indexing. Students learn how to access individual characters using these indices. The concept of string length and how to measure it using the built-in `len()` function is then explained. Next, the lecture demonstrates two ways to traverse (iterate over) strings: directly looping through characters and looping through indices using a `for` loop with `range()`. Lastly, the lecture explains various ways of defining string literals using single quotes, double quotes, triple single quotes, and triple double quotes, including how to write multi-line strings and handle quotes inside strings.

Section 2: Key Concepts and Explanations

- What is a String?

- A string is a collection of characters enclosed in quotes (`'...'` or `"..."`) in Python.
- It can contain alphabets (uppercase and lowercase), numbers, special characters (`@, ., etc.`), spaces, or any keyboard character.
- Strings represent text data and are a fundamental data type in programming.

- String Representation in Python

- Internally, a string is stored as a sequence (similar to an array) where each character occupies a position (index).
- Indices start at 0 for the first character and increase by 1 for each subsequent character (e.g., `H=0, e=1, l=2, ...`).
- Python also supports negative indices, starting at -1 for the last character, going backward (e.g., `o=-1, l=-2, ...`).

- Accessing characters uses these indices, for example `s1` or `s1[-1]` returns the first and last character.

- Measuring String Length

- The length of a string is the total number of characters it contains, including spaces and special characters.

- The `len()` function returns the length of any sequence including strings.

- For example, `len("Hello")` returns 5.

- Traversing a String

- Traversing means visiting each character in the string one by one.

- Method 1: Direct iteration over the string in a `for` loop: `for x in s1:`

Each iteration assigns one character to `x`.

- Method 2: Use a `for` loop with indices using `range(len(s1))` and access characters by index: `s1[i]`.

- Both methods print or manipulate all characters in order.

- String Literals

- Strings can be defined using:

- Single quotes: `'Hello'`

- Double quotes: `"Hello"`

- Triple single quotes: `'''Hello'''` (useful for multi-line strings)

- Triple double quotes: `"""Hello"""` (also supports multi-line)

- Triple quotes can be used for multi-line text or comments (docstrings).

- To include apostrophes or single quotes inside a string, use double quotes to enclose the string: `"Clark's"`.

- Similarly, to include double quotes inside a string, use single quotes to enclose it: `'She said "Hi"'`.

Section 3: Example Code and Use Cases

String declaration

```
s1 = 'Hello'
```

Creates a string variable `s1` with the text "Hello".

Accessing characters using indexing

```
print(s1[0])    # Output: H
print(s1[-1])   # Output: o
print(s1[4])    # Output: o
print(s1[-5])   # Output: H
```

Accesses characters using positive and negative indices.

Getting length of a string

```
print(len(s1))  # Output: 5
```

Uses built-in function `len()` to get string length.

Traversing string - Method 1: Direct iteration

```
for x in s1:
    print(x)
```

Prints each character in `s1` one by one.

Traversing string - Method 2: Using indices

```
for i in range(len(s1)):
    print(s1[i])
```

Prints characters by accessing indices in a `for` loop range from 0 to length-1.

String literals using different quotes

```
s2 = "Hello"
s3 = '''Hello'''
```

```
s4 = """Hello  
How are you?"""
```

Demonstrates single, double, and triple quoted strings including multi-line string.

Handling apostrophes and quotes inside strings

```
s5 = "Clark's"  
s6 = 'She said "Hello"'
```

Shows using different outer quotes to include quotes/apostrophes inside strings.

Section 4: Key Takeaways

- String is a sequence of characters enclosed in quotes, capable of holding alphabets, numbers, symbols, and spaces.
- Python strings are sequences with both positive and negative indices for accessing characters.
- Use `len()` function to get the length of a string efficiently.
- Strings can be traversed using a `for` loop directly over characters or by index via `range()`.
- Multiple ways to define strings exist (single, double, triple quotes), enabling multi-line strings and embedded quotes inside text.
- Understanding string indexing and traversal is fundamental for string manipulation tasks.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.





String Introduction



String Introduction

String Introduction

What is a String ?

String Introduction

What is a String ?

String Representation

String Introduction

What is a String ?

String Representation

String Length

String Introduction

What is a String ?

String Representation

String Length

Traversing a String

String Introduction

What is a String ?

String Representation

String Length

Traversing a String

String Literals

What is a String ?

What is a String ?

S1 = 'Hello'

What is a String ?

S1 = 'Hello'



What is a String ?

S1 = 'Hello'

S2 = '9784909090'



What is a String ?

S1 = 'Hello'

S2 = '9784909090'

S3 = 'my.id123@gmail.com'



What is a String ?

S1 = 'Hello'

S2 = '9784909090'

S3 = 'my.id123@gmail.com'

S4 = 'My Book'



What is a String ?

S1 = 'Hello'

S2 = '9784909090'

S3 = 'my.id123@gmail.com'

S4 = 'My Book'



What is a String ?

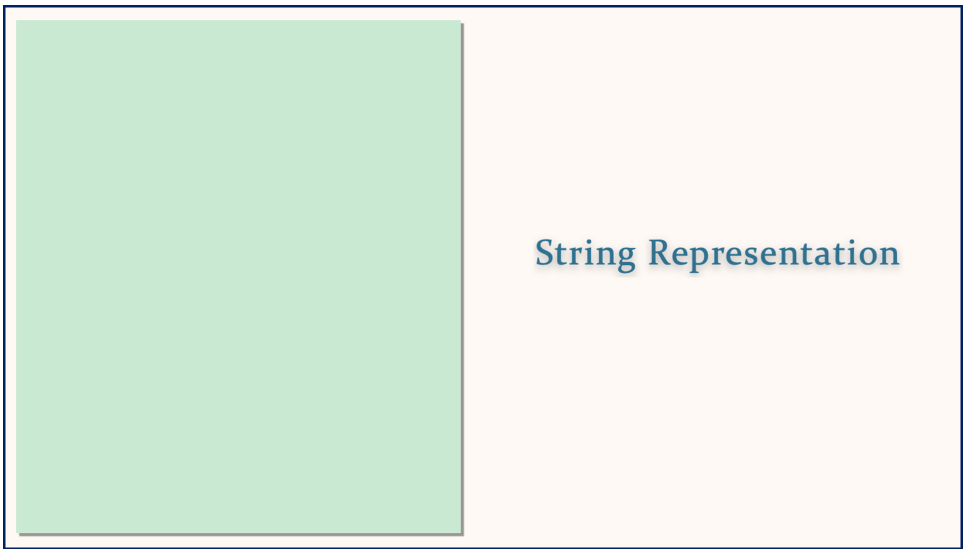
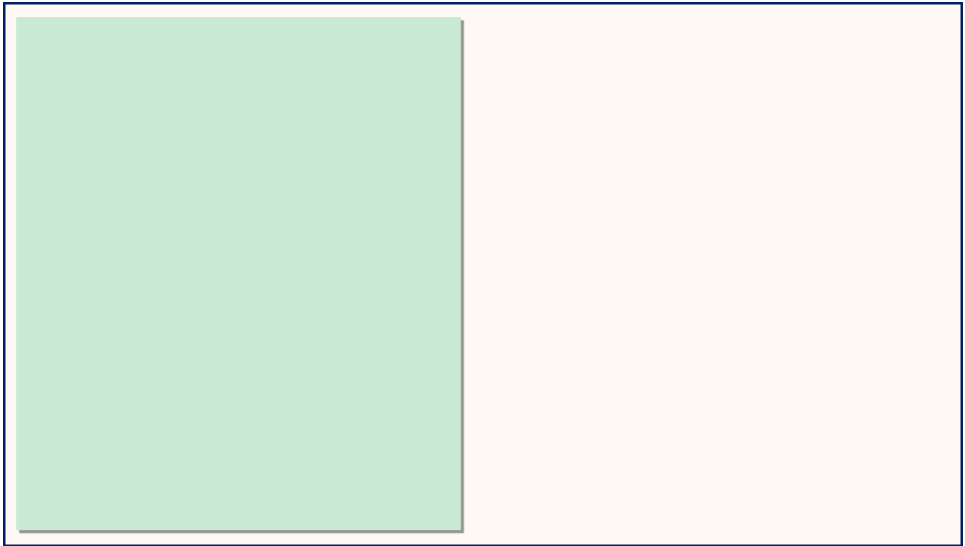
S1 = 'Hello'

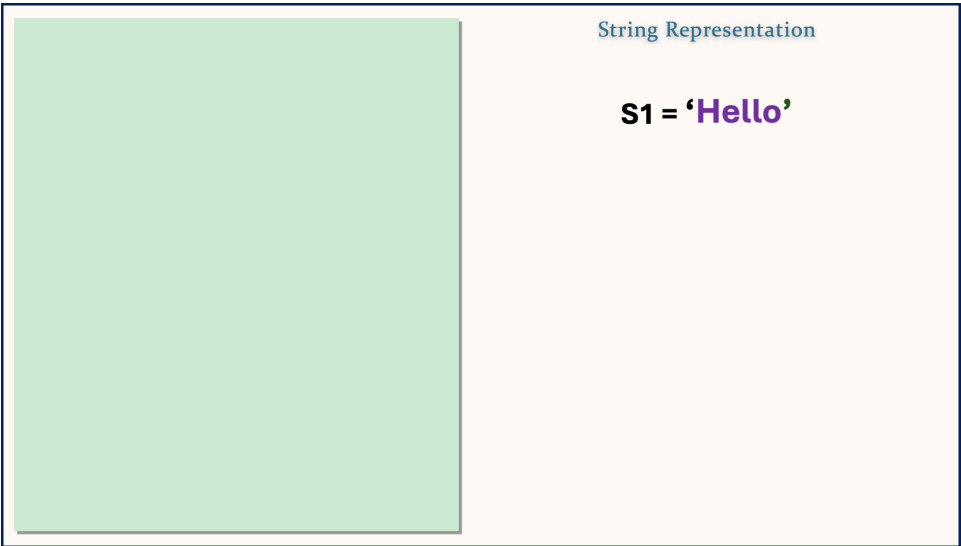
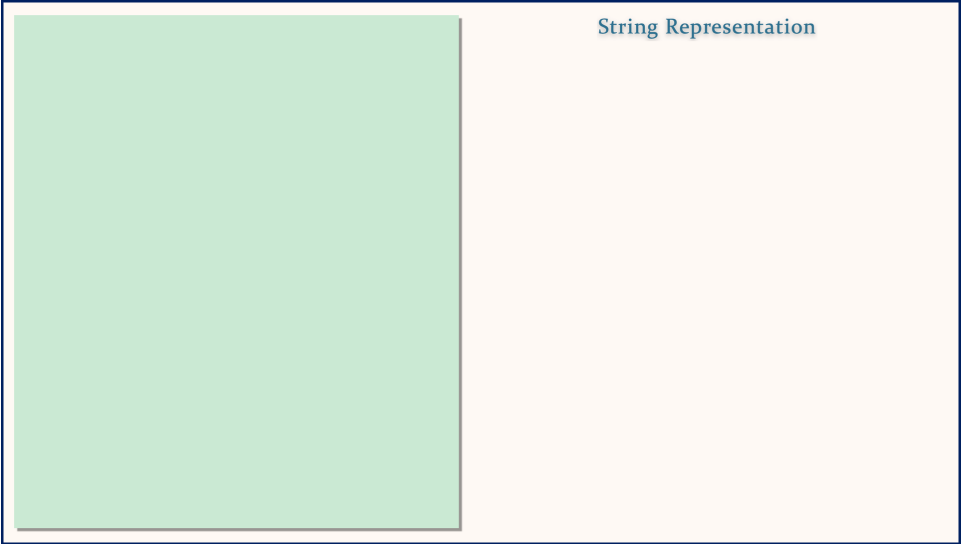
S2 = '9784909090'

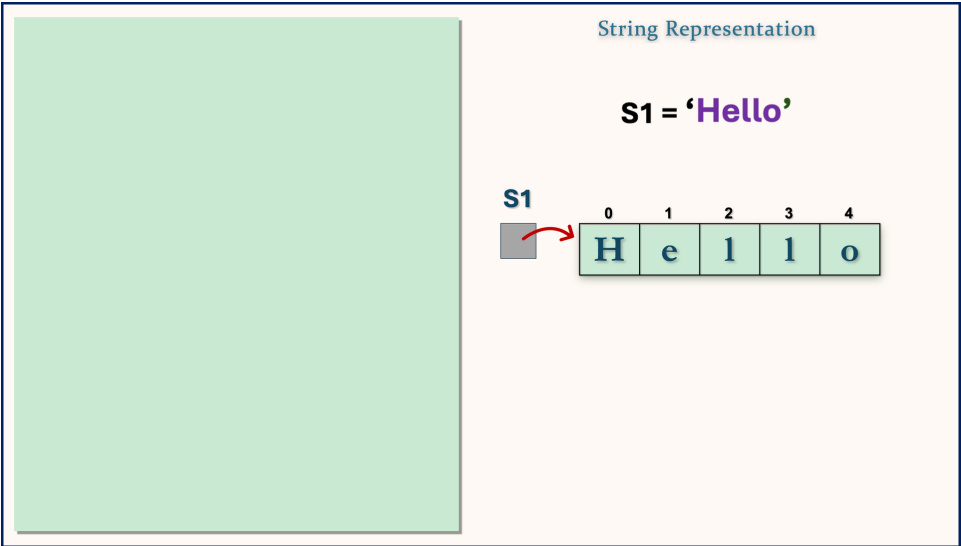
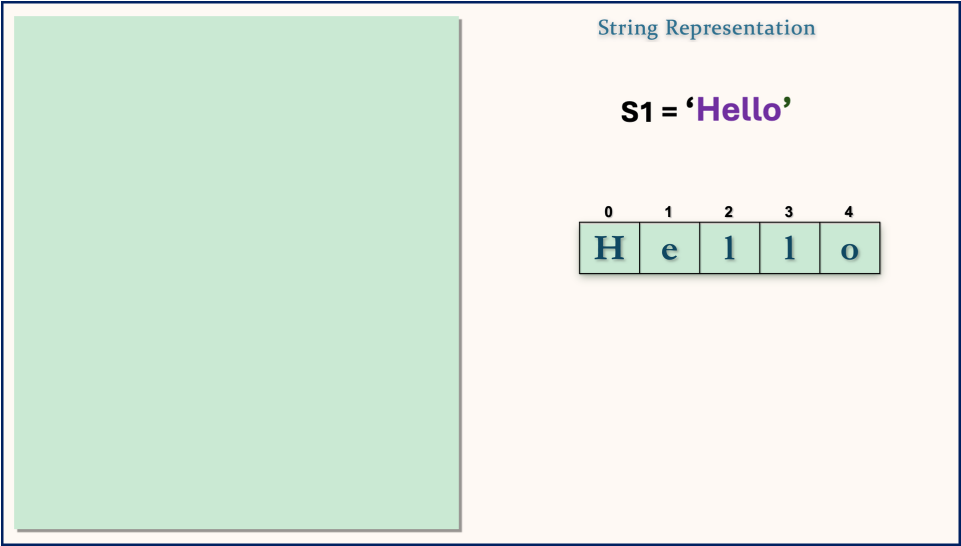
S3 = 'my.id123@gmail.com'

S4 = 'My Book'



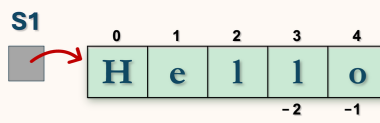






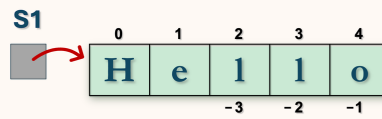
String Representation

S1 = 'Hello'



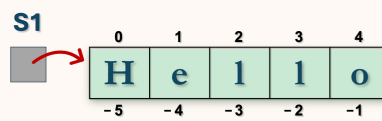
String Representation

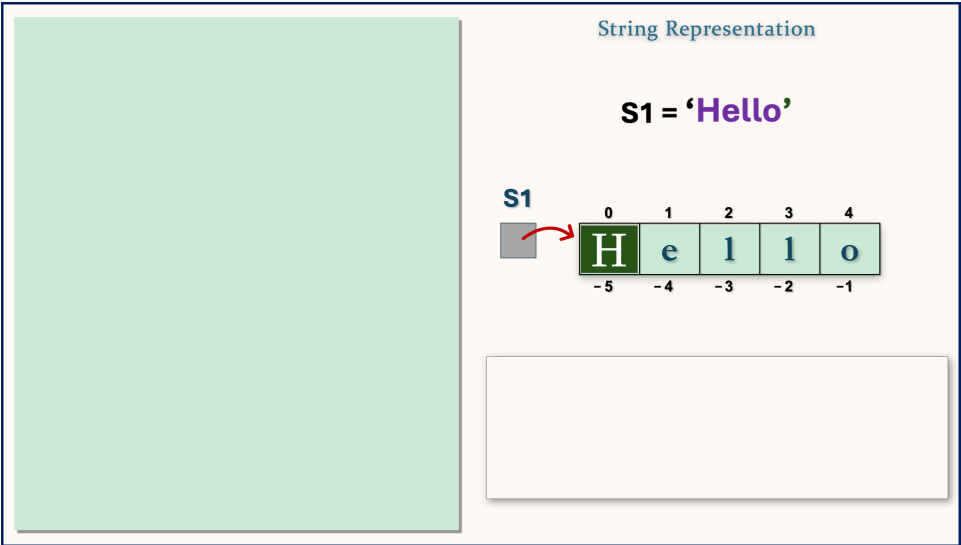
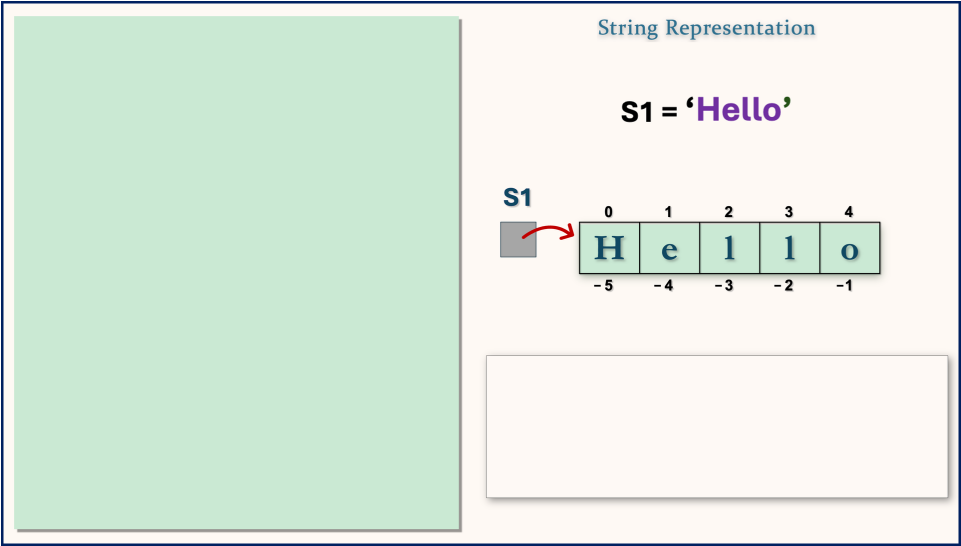
S1 = 'Hello'



String Representation

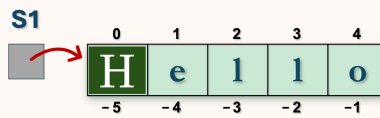
S1 = 'Hello'





String Representation

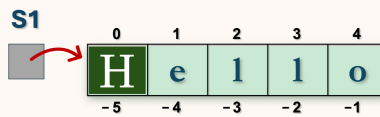
S1 = 'Hello'



```
H ← print(s1[0])
```

String Representation

S1 = 'Hello'

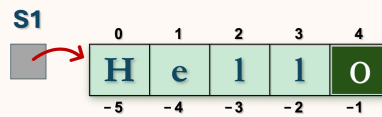


```
H ← print(s1[0])
```

```
H ← print(s1[-5])
```


String Representation

S1 = 'Hello'

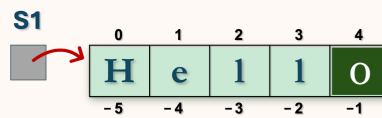


```
H ← print(s1[0])
```

```
H ← print(s1[-5])
```

String Representation

S1 = 'Hello'



```
H ← print(s1[0])    O ← print(s1[4])
```

```
H ← print(s1[-5])
```

String Representation

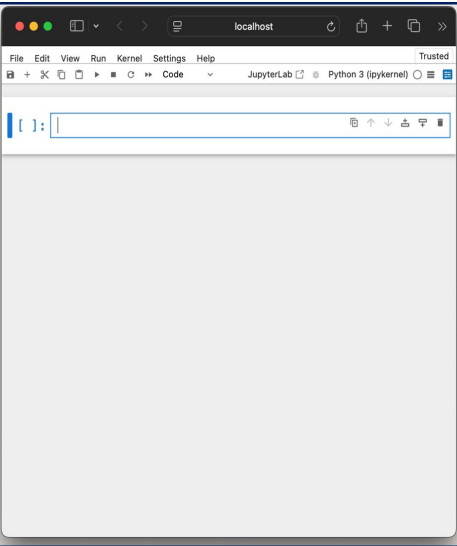
S1 = 'Hello'

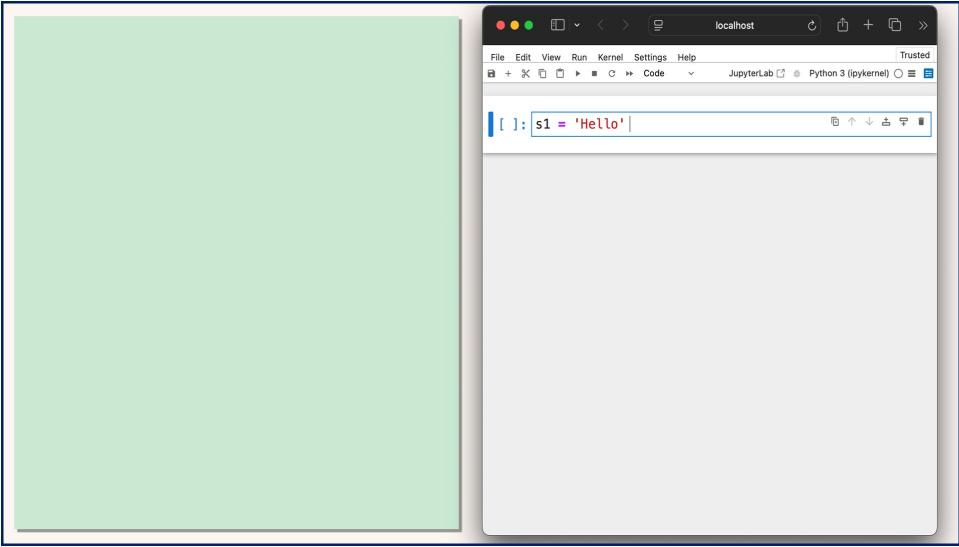
S1

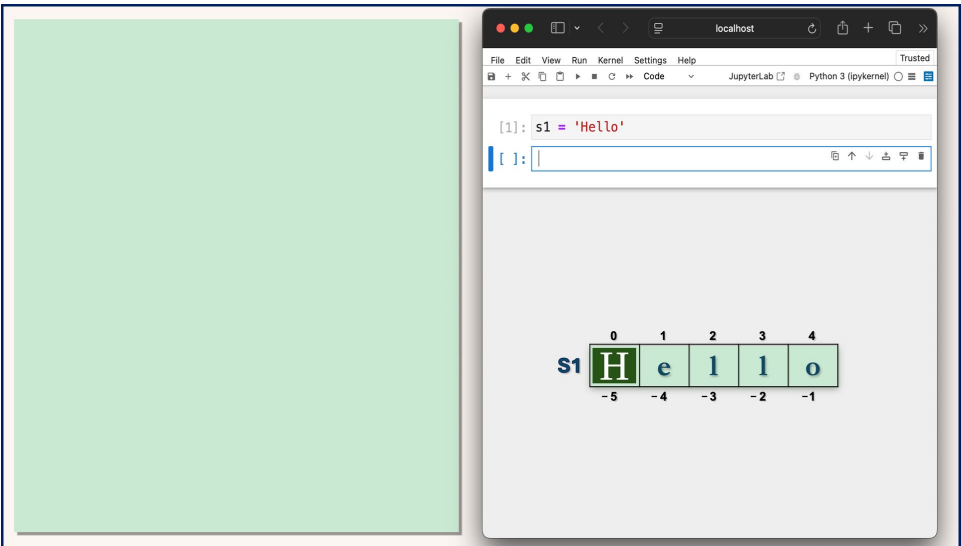
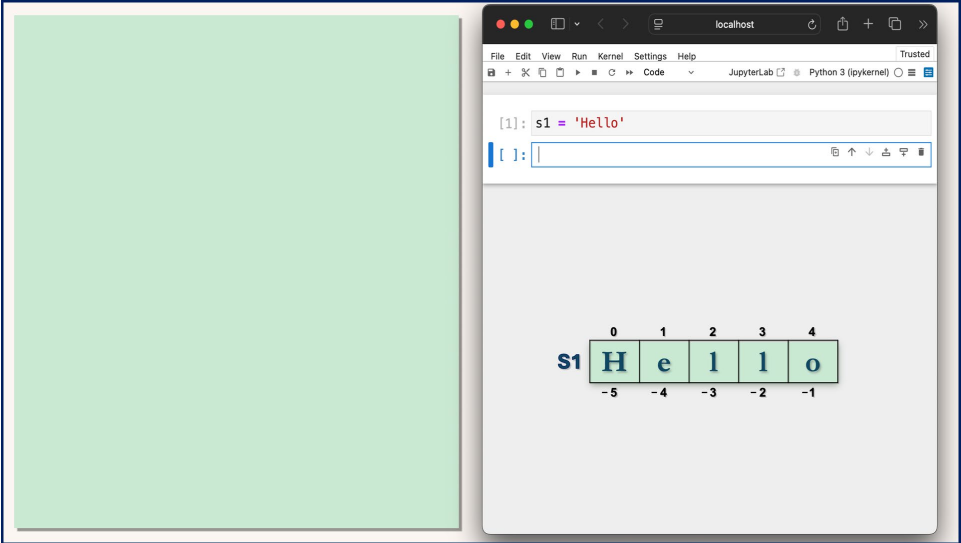
	0	1	2	3	4
	H	e	l	l	O
	-5	-4	-3	-2	-1

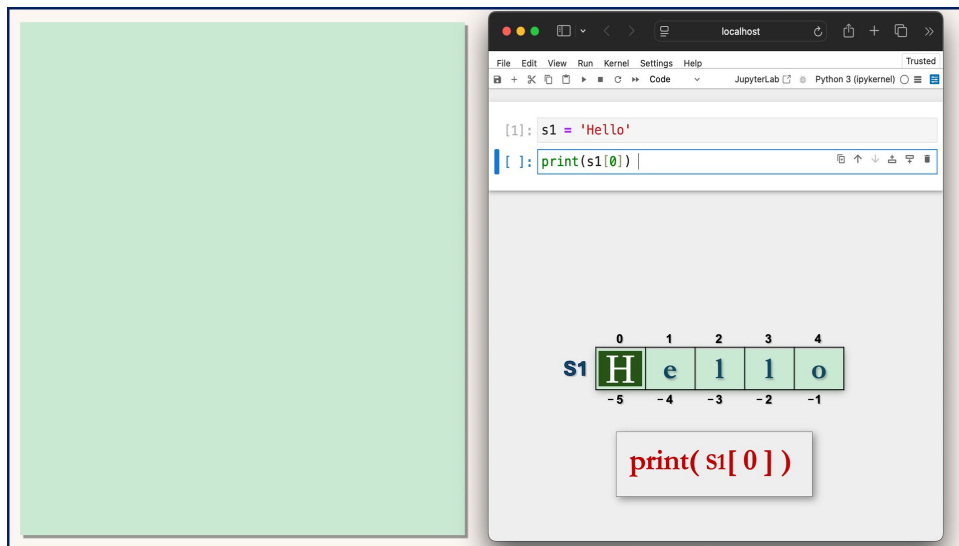
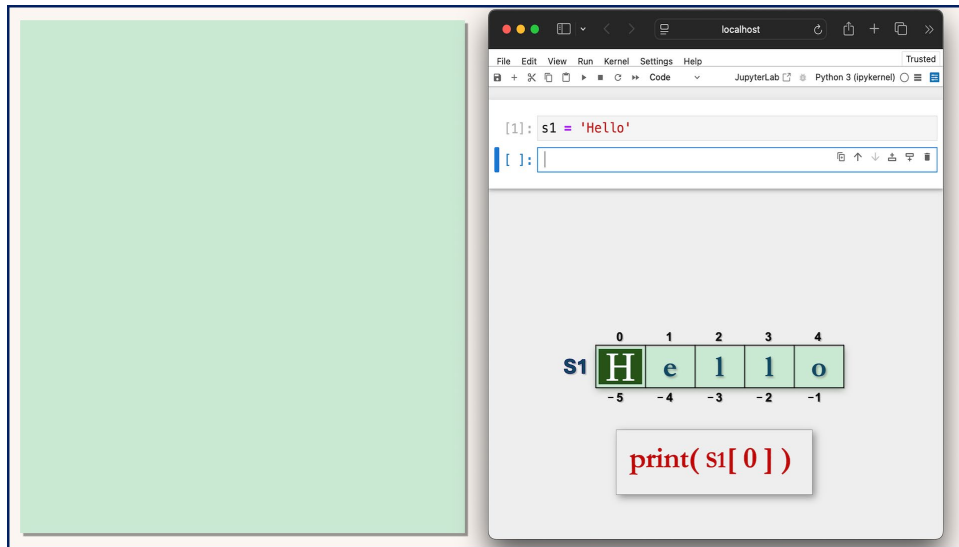
H ← print(s1[0]) O ← print(s1[4])

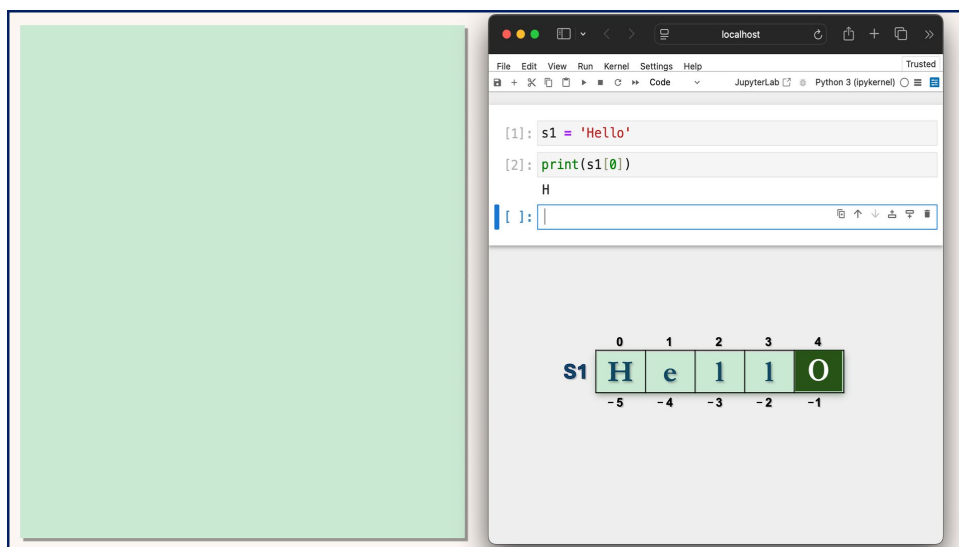
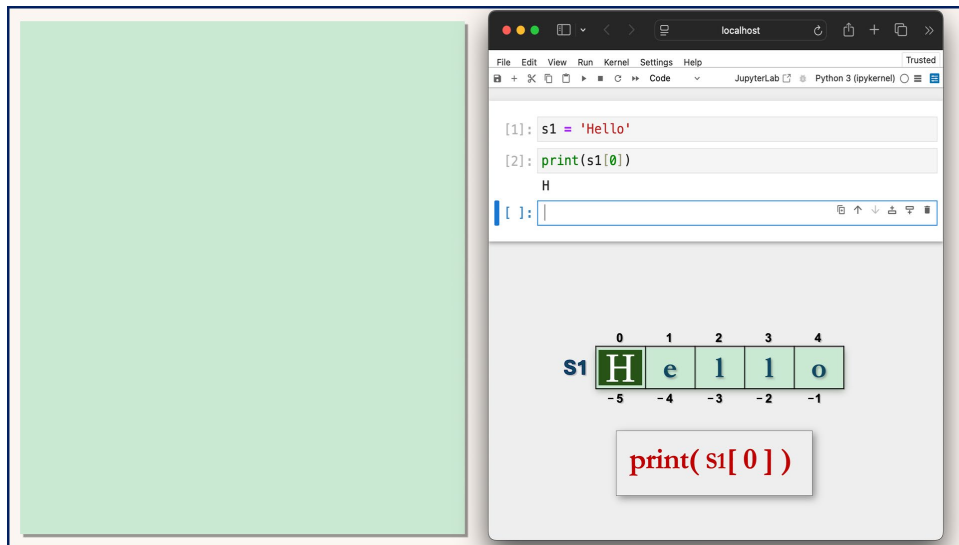
H ← print(s1[-5]) O ← print(s1[-1])

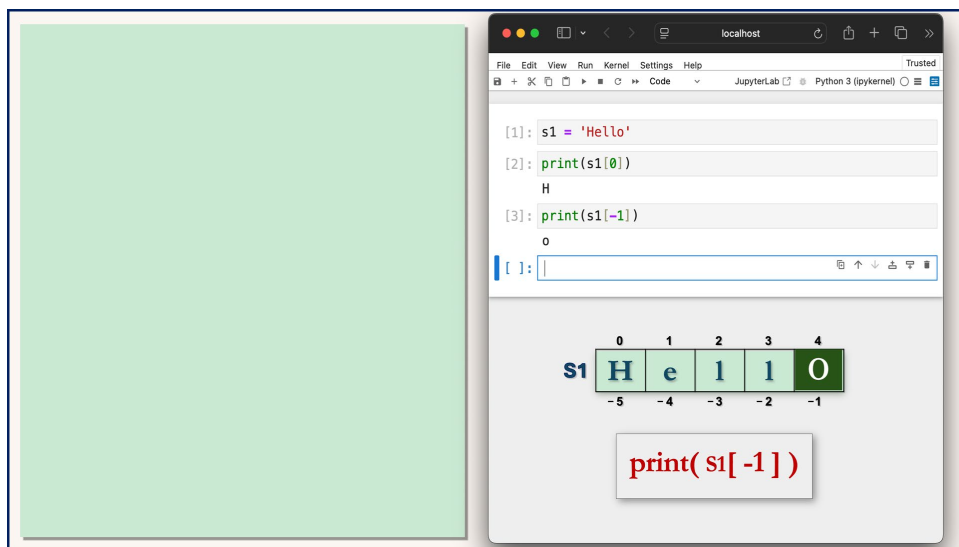
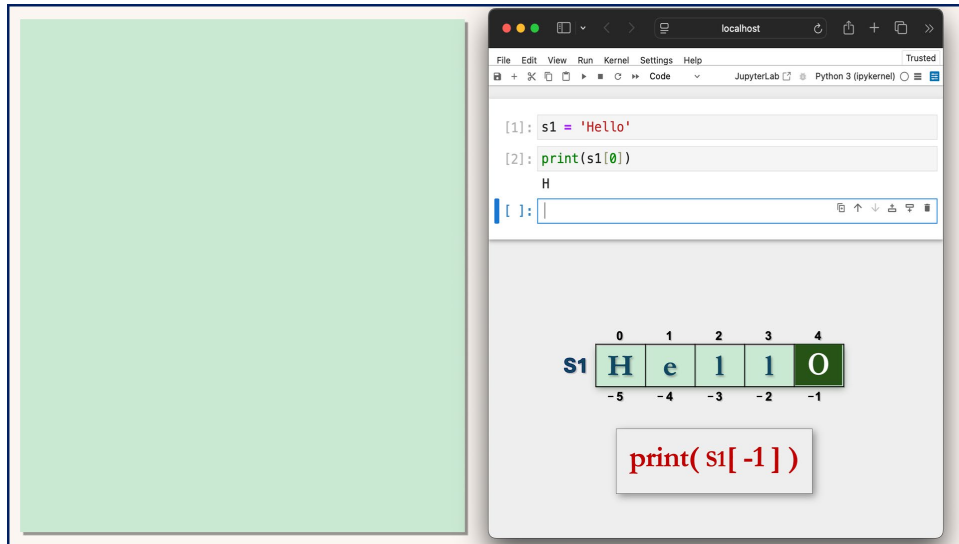




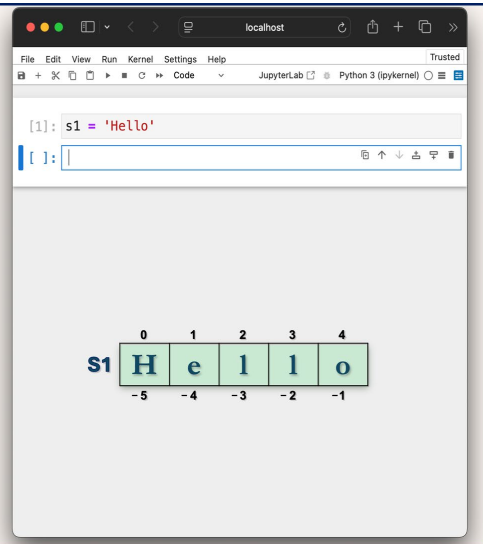








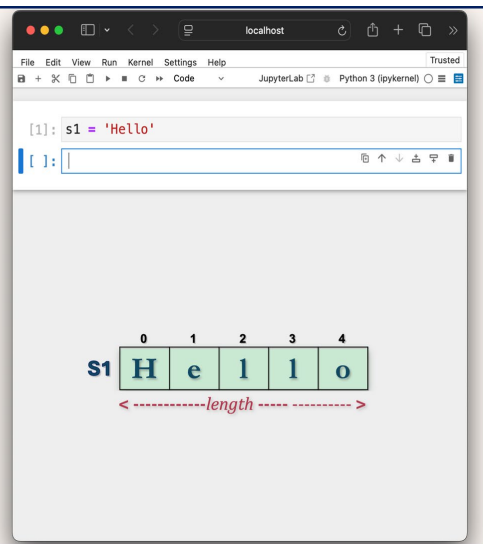
String Length



The image shows a JupyterLab interface with a code cell containing the Python code `s1 = 'Hello'`. Below the code cell, the variable `s1` is displayed with its value `'Hello'`. The string is visualized as a sequence of characters in boxes, with indices 0 through 4 above each character and indices -5 through -1 below each character.

	0	1	2	3	4
s1	H	e	l	l	o
	-5	-4	-3	-2	-1

String Length

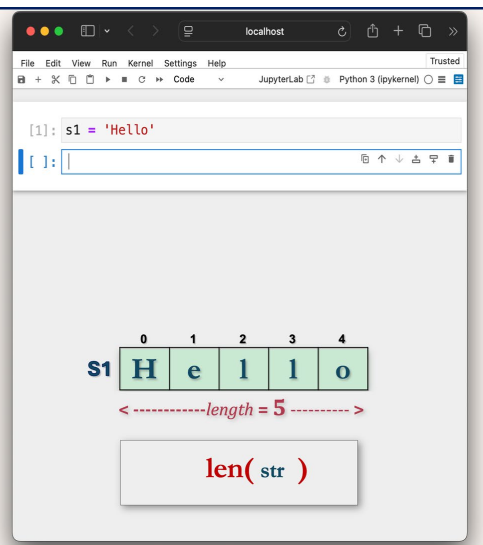


The image shows a JupyterLab interface with a code cell containing the Python code `s1 = 'Hello'`. Below the code cell, the variable `s1` is displayed with its value `'Hello'`. The string is visualized as a sequence of characters in boxes, with indices 0 through 4 above each character and indices -5 through -1 below each character. A red dashed line with arrows at both ends is drawn below the string, labeled `length`, indicating the length of the string.

	0	1	2	3	4
s1	H	e	l	l	o
	-5	-4	-3	-2	-1

length

String Length



The image shows a JupyterLab interface with a code cell and an output cell. The code cell contains the line `s1 = 'Hello'`. The output cell displays a diagram of the string 'Hello' stored in memory. The string is represented as a sequence of characters in a table, with indices 0 through 4 above each character: 'H' at index 0, 'e' at index 1, 'l' at index 2, 'l' at index 3, and 'o' at index 4. Below the table, a red dashed line indicates the length of the string, labeled `length = 5`. At the bottom of the output cell, a button labeled `len(str)` is shown.

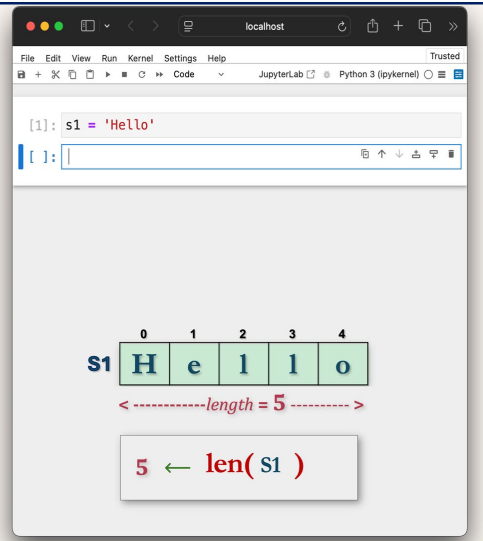
```
[1]: s1 = 'Hello'
```

	0	1	2	3	4
s1	H	e	l	l	o

< -----length = 5 ----- >

`len(str)`

String Length



The image shows a JupyterLab interface with a code editor and a console. The code editor contains the following code:

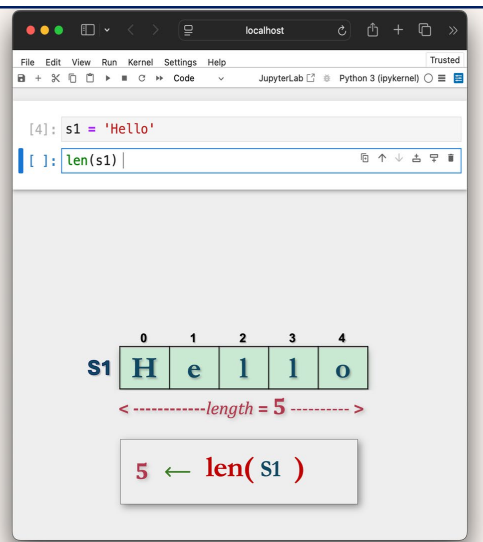
```
[1]: s1 = 'Hello'
```

The console shows the output of the code:

```
[ ]: 
```

Below the console, a diagram illustrates the string 'Hello' as a sequence of characters in memory. The characters are 'H', 'e', 'l', 'l', 'o', each in a green box. Above the boxes are indices 0, 1, 2, 3, 4. A red dashed line with arrows at both ends spans the five boxes, with the text 'length = 5' in red. Below this, a box contains the code '5 ← len(s1)' in red, with a green arrow pointing from the number 5 to the variable s1.

String Length



The image shows a JupyterLab interface with a code editor and a console. The code editor contains the following code:

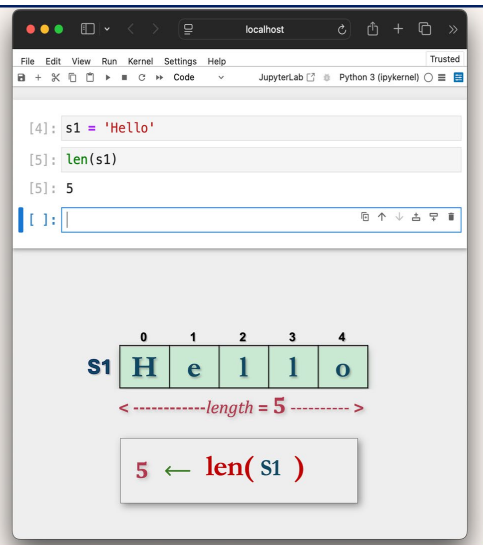
```
[4]: s1 = 'Hello'
```

The console shows the output of the code:

```
[ ]: len(s1)
```

Below the console, a diagram illustrates the string 'Hello' as a sequence of characters in memory. The characters are 'H', 'e', 'l', 'l', 'o', each in a green box. Above the boxes are indices 0, 1, 2, 3, 4. A red dashed line with arrows at both ends spans the five boxes, with the text 'length = 5' in red. Below this, a box contains the code '5 ← len(s1)' in red, with a green arrow pointing from the number 5 to the variable s1.

String Length



The screenshot shows a JupyterLab window with the following content:

```
[4]: s1 = 'Hello'
[5]: len(s1)
[5]: 5
```

Below the code, a diagram illustrates the string 'Hello' stored in memory. The string is represented as a sequence of characters in boxes, indexed from 0 to 4:

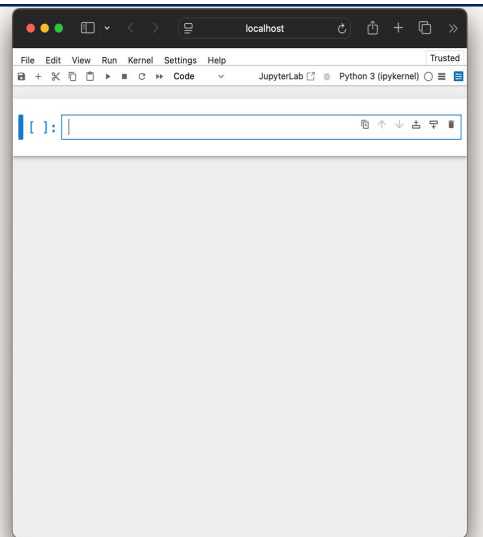
	0	1	2	3	4
s1	H	e	l	l	o

A red dashed line with arrows at both ends spans the width of the five character boxes, with the text `length = 5` written in red above it.

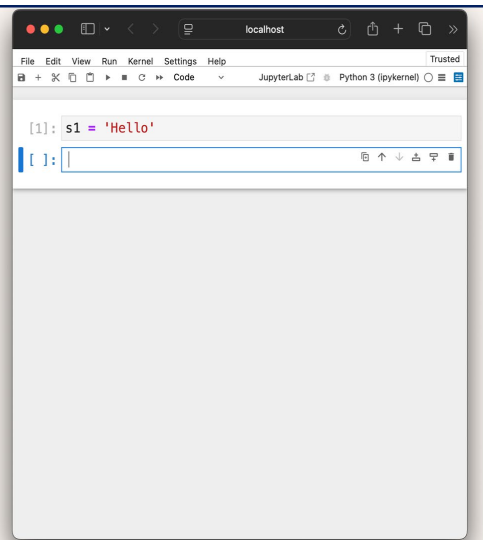
Below the diagram, a box contains the code `5 ← len(s1)` in red, indicating the result of the `len()` function call.

Traversing a String

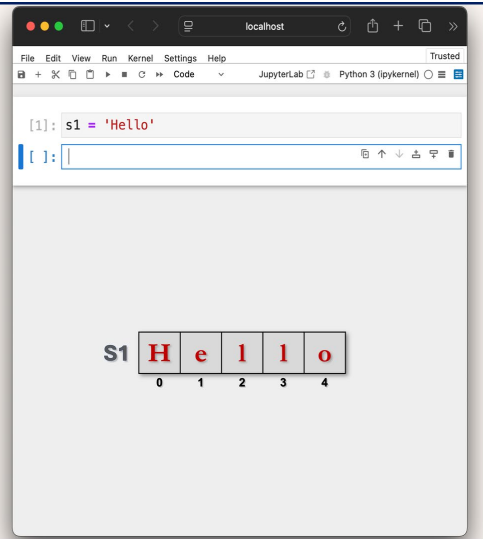
Traversing a String



Traversing a String



Traversing a String



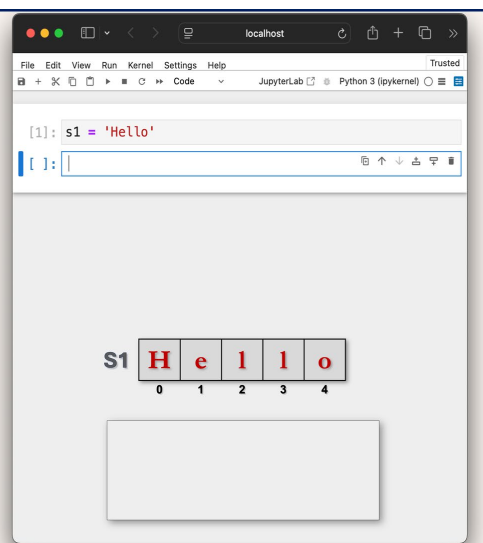
The image shows a JupyterLab interface on a localhost. The top bar includes a menu (File, Edit, View, Run, Kernel, Settings, Help) and a status bar (Trusted). Below the menu is a toolbar with icons for file operations and code execution. The main area contains a code cell with the text `[1]: s1 = 'Hello'` and an output cell. The output cell displays a diagram of a string in memory. The string is represented as a sequence of characters in boxes, with indices 0 through 4 below them. The characters are 'H', 'e', 'l', 'l', 'o'.

```
[1]: s1 = 'Hello'
```

s1

H	e	l	l	o
0	1	2	3	4

Traversing a String



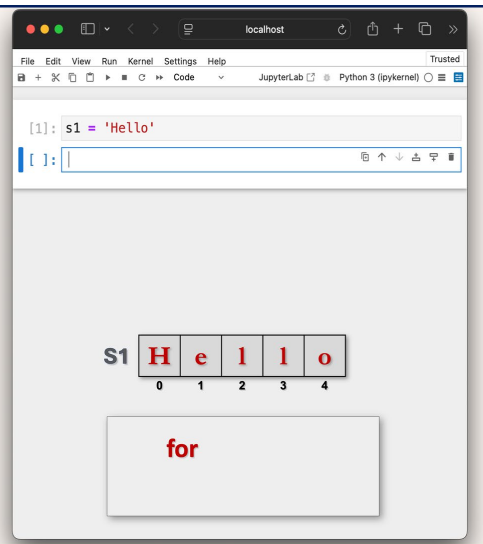
The image shows a JupyterLab interface on a localhost, identical to the one above. The code cell contains `[1]: s1 = 'Hello'`. The output cell displays the same string memory diagram as above. Below the diagram is an empty rectangular box, likely intended for additional output or a diagram illustrating string traversal.

```
[1]: s1 = 'Hello'
```

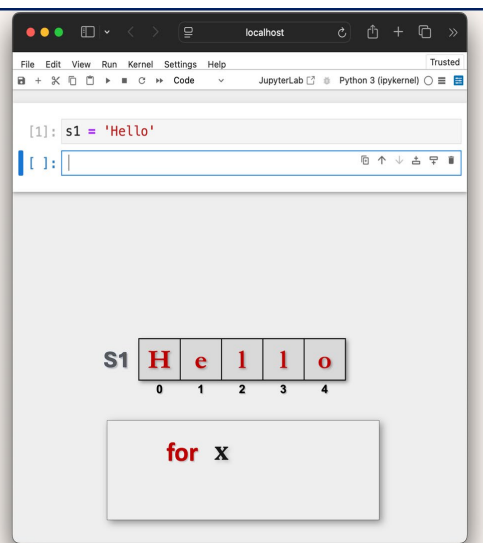
s1

H	e	l	l	o
0	1	2	3	4

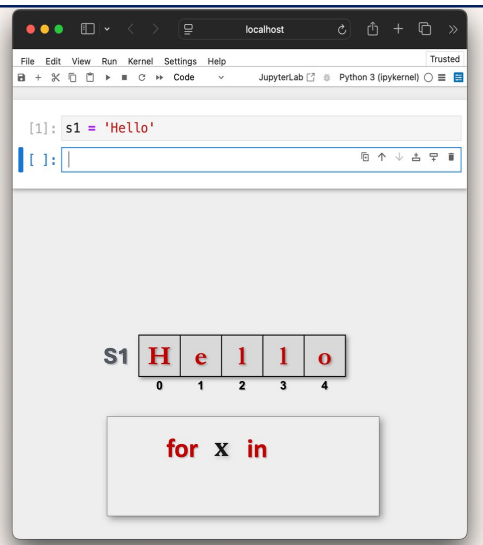
Traversing a String



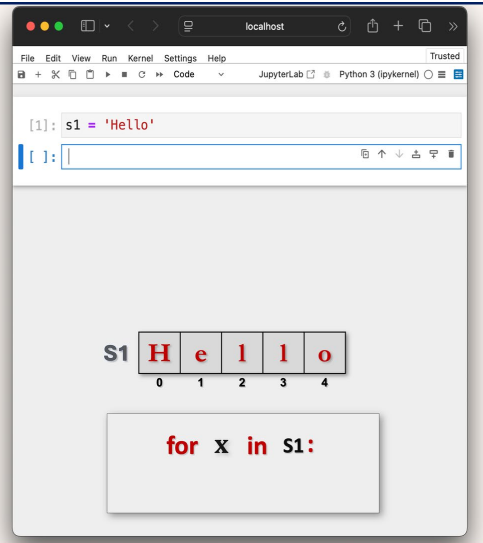
Traversing a String



Traversing a String



Traversing a String



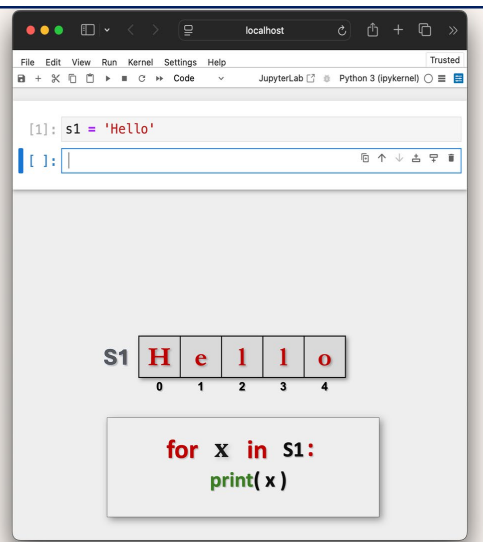
```
[1]: s1 = 'Hello'
```

S1

H	e	l	l	o
0	1	2	3	4

```
for x in s1:
```

Traversing a String



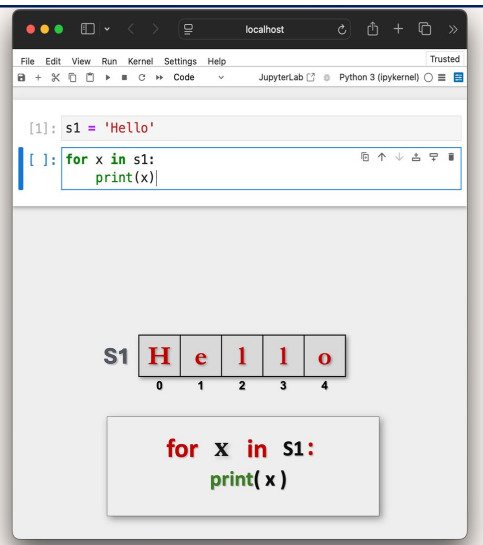
```
[1]: s1 = 'Hello'
```

S1

H	e	l	l	o
0	1	2	3	4

```
for x in s1:  
    print(x)
```


Traversing a String



The JupyterLab interface shows the following code in the code editor:

```
[1]: s1 = 'Hello'
```

```
[ ]: for x in s1:  
      print(x)
```

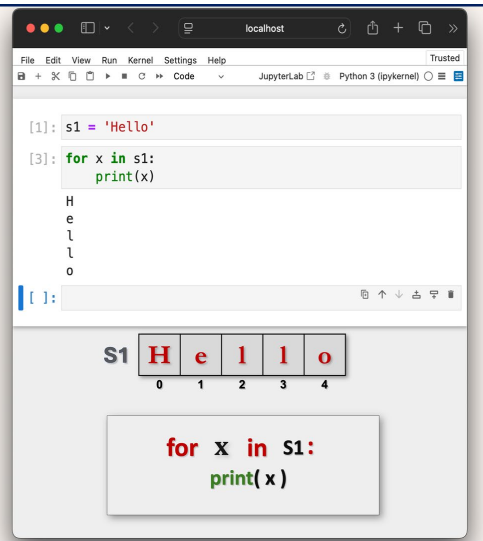
Below the code editor, a memory diagram illustrates the string 'Hello' stored in variable `s1`. The string is represented as a sequence of characters in memory cells, indexed from 0 to 4:

Index	0	1	2	3	4
Character	H	e	l	l	o

Below the memory diagram, a code snippet shows the loop structure:

```
for x in s1:  
    print(x)
```

Traversing a String



The JupyterLab interface shows the following code in the code editor:

```
[1]: s1 = 'Hello'
```

```
[3]: for x in s1:  
      print(x)
```

Below the code editor, the output of the code is displayed as a vertical list of characters:

```
H  
e  
l  
l  
o
```

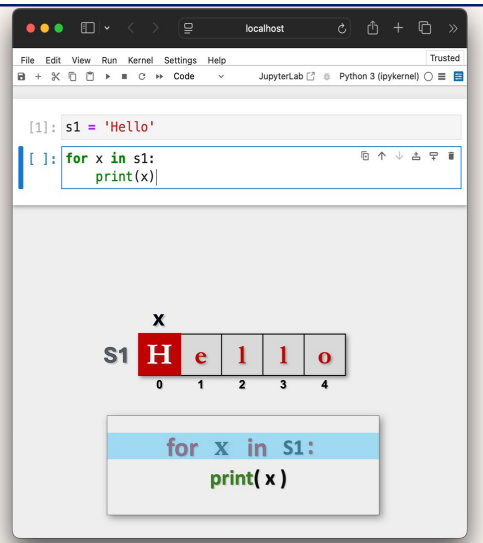
Below the output, a memory diagram illustrates the string 'Hello' stored in variable `s1`. The string is represented as a sequence of characters in memory cells, indexed from 0 to 4:

Index	0	1	2	3	4
Character	H	e	l	l	o

Below the memory diagram, a code snippet shows the loop structure:

```
for x in s1:  
    print(x)
```

Traversing a String



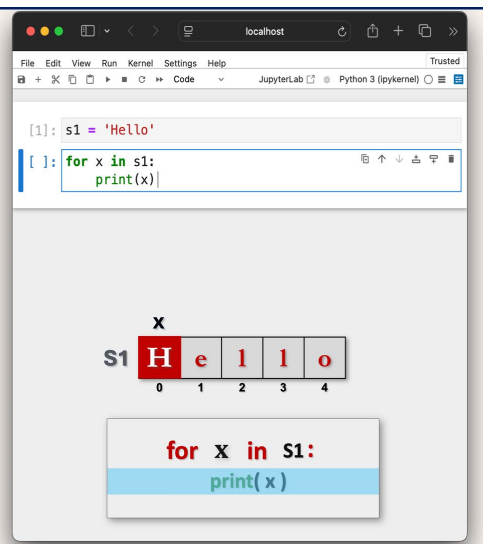
The image shows a JupyterLab interface with a code editor and a diagram. The code editor contains the following Python code:

```
[1]: s1 = 'Hello'
[ ]: for x in s1:
      print(x)
```

Below the code editor, there is a diagram illustrating the string 'Hello' as a sequence of characters in memory. The string is represented as a horizontal array of five boxes, each containing a character: 'H', 'e', 'l', 'l', 'o'. Above the first box ('H') is the variable 'x', and to the left of the array is the label 's1'. Below each box is its corresponding index: 0, 1, 2, 3, 4. A blue box highlights the first box ('H'). Below the diagram, there is a code snippet in a light blue box:

```
for x in s1:
    print(x)
```

Traversing a String



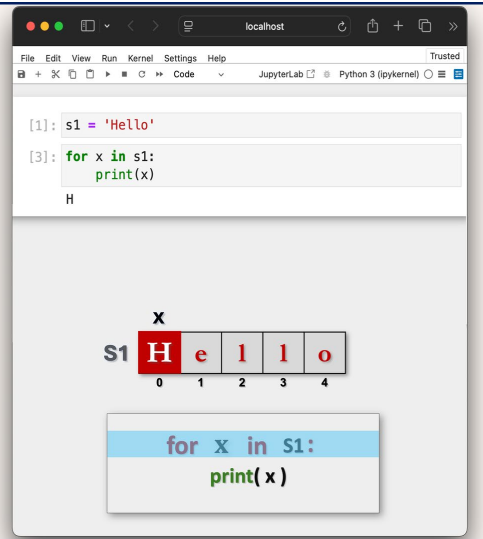
The image shows a JupyterLab interface with a code editor and a diagram. The code editor contains the following Python code:

```
[1]: s1 = 'Hello'
[ ]: for x in s1:
      print(x)
```

Below the code editor, there is a diagram illustrating the string 'Hello' as a sequence of characters in memory. The string is represented as a horizontal array of five boxes, each containing a character: 'H', 'e', 'l', 'l', 'o'. Above the first box ('H') is the variable 'x', and to the left of the array is the label 's1'. Below each box is its corresponding index: 0, 1, 2, 3, 4. A blue box highlights the first box ('H'). Below the diagram, there is a code snippet in a light blue box:

```
for x in s1:
    print(x)
```

Traversing a String



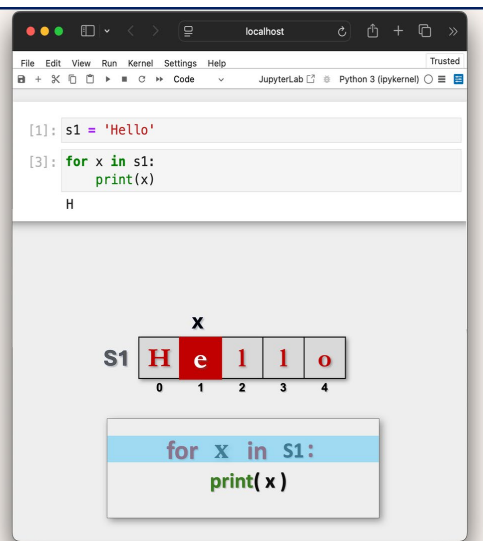
The image shows a JupyterLab interface with a code editor and a diagram. The code editor contains the following code:

```
[1]: s1 = 'Hello'
[3]: for x in s1:
      print(x)
```

The output of the code is 'H'. Below the code editor, there is a diagram illustrating the string 'Hello' as a sequence of characters in memory. The string is represented as a list of characters: 'H', 'e', 'l', 'l', 'o'. Each character is in a red box. Above the first character 'H' is the variable 'x'. Below the characters are their indices: 0, 1, 2, 3, 4. To the left of the first character is the label 's1'. Below the diagram, there is a code block with the following code:

```
for x in s1:
    print(x)
```

Traversing a String



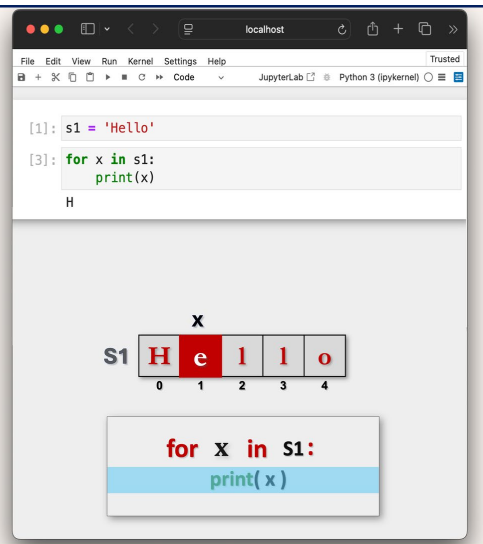
The image shows a JupyterLab interface with a code editor and a diagram. The code editor contains the following code:

```
[1]: s1 = 'Hello'
[3]: for x in s1:
      print(x)
```

The output of the code is 'H'. Below the code editor, there is a diagram illustrating the string 'Hello' as a sequence of characters in memory. The string is represented as a list of characters: 'H', 'e', 'l', 'l', 'o'. Each character is in a red box. Above the first character 'H' is the variable 'x'. Below the characters are their indices: 0, 1, 2, 3, 4. To the left of the first character is the label 's1'. Below the diagram, there is a code block with the following code:

```
for x in s1:
    print(x)
```

Traversing a String



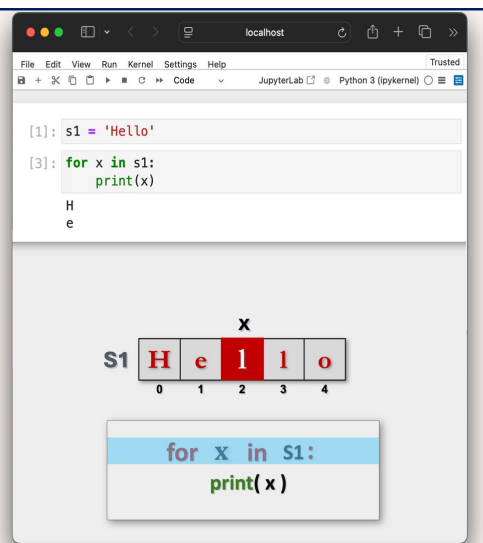
The JupyterLab interface shows a code cell with the following Python code:

```
[1]: s1 = 'Hello'
[3]: for x in s1:
      print(x)
```

The output of the code is the character 'H'. Below the code cell, a diagram illustrates the string 'Hello' as a sequence of characters in boxes, indexed from 0 to 4. The character 'e' at index 1 is highlighted in red. A variable 'x' is shown above the 'e' box. Below the diagram, a code snippet shows the loop structure:

```
for x in s1:
    print(x)
```

Traversing a String



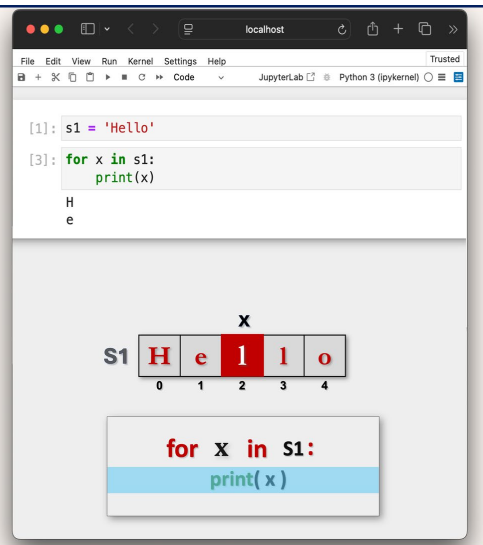
The JupyterLab interface shows a code cell with the following Python code:

```
[1]: s1 = 'Hello'
[3]: for x in s1:
      print(x)
```

The output of the code is the characters 'H' and 'e'. Below the code cell, a diagram illustrates the string 'Hello' as a sequence of characters in boxes, indexed from 0 to 4. The character 'l' at index 2 is highlighted in red. A variable 'x' is shown above the 'l' box. Below the diagram, a code snippet shows the loop structure:

```
for x in s1:
    print(x)
```

Traversing a String



```
[1]: s1 = 'Hello'
```

```
[3]: for x in s1:  
    print(x)
```

H
e

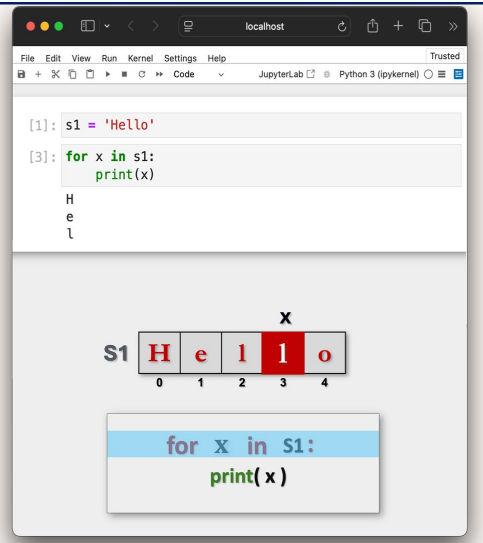
s1

	H	e	l	l	o
	0	1	2	3	4

x

```
for x in s1:  
    print(x)
```

Traversing a String



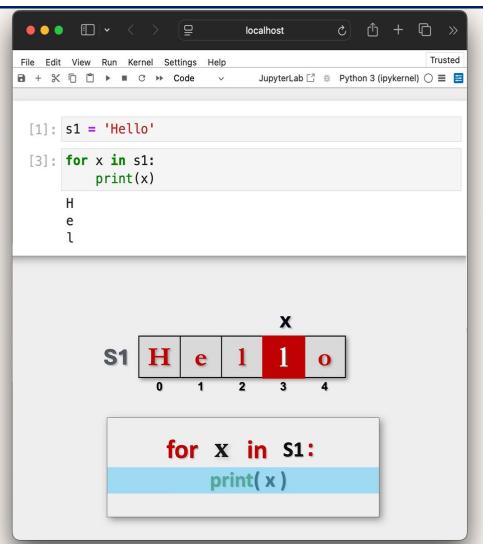
The image shows a JupyterLab interface with a code editor and an output area. The code editor contains the following Python code:

```
[1]: s1 = 'Hello'
[3]: for x in s1:
      print(x)
```

The output area displays the characters 'H', 'e', and 'l' on separate lines. Below the code editor, there is a diagram illustrating the string 'Hello' as a sequence of characters in memory. The string is represented as a sequence of boxes: 'H', 'e', 'l', 'l', 'o'. The first 'l' is highlighted in red. Above the first 'l' is the variable 'x'. Below the boxes are indices 0, 1, 2, 3, 4. Below the diagram, there is a code block with the following Python code:

```
for x in s1:
    print(x)
```

Traversing a String



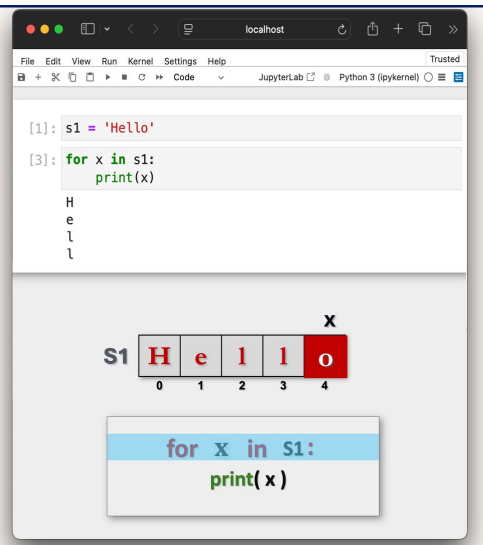
The image shows a JupyterLab interface with a code editor and an output area. The code editor contains the following Python code:

```
[1]: s1 = 'Hello'
[3]: for x in s1:
      print(x)
```

The output area displays the characters 'H', 'e', and 'l' on separate lines. Below the code editor, there is a diagram illustrating the string 'Hello' as a sequence of characters in memory. The string is represented as a sequence of boxes: 'H', 'e', 'l', 'l', 'o'. The first 'l' is highlighted in red. Above the first 'l' is the variable 'x'. Below the boxes are indices 0, 1, 2, 3, 4. Below the diagram, there is a code block with the following Python code:

```
for x in s1:
    print(x)
```

Traversing a String

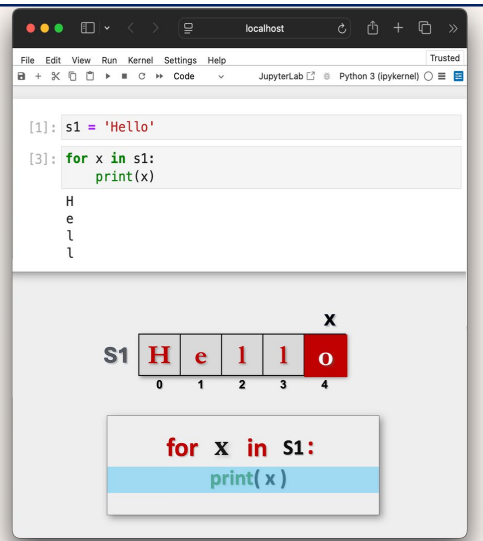


The image shows a JupyterLab interface with a code editor and an output area. The code editor contains the following Python code:

```
[1]: s1 = 'Hello'
[3]: for x in s1:
      print(x)
```

The output area displays the characters 'H', 'e', 'l', 'l' on separate lines. Below the output, there is a diagram illustrating the string 'Hello' as a sequence of characters in memory. The string is represented as a list of boxes: 'H' (index 0), 'e' (index 1), 'l' (index 2), 'l' (index 3), and 'o' (index 4). A variable 'x' is shown pointing to the 'o' character at index 4. Below the diagram, a code block shows the loop: `for x in s1:` followed by `print(x)`.

Traversing a String

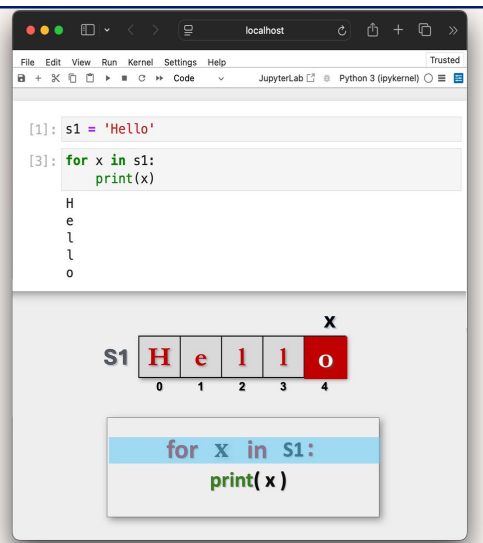


The image shows a JupyterLab interface with a code editor and an output area. The code editor contains the following Python code:

```
[1]: s1 = 'Hello'
[3]: for x in s1:
      print(x)
```

The output area displays the characters 'H', 'e', 'l', 'l' on separate lines. Below the output, there is a diagram illustrating the string 'Hello' as a sequence of characters in memory. The string is represented as a list of boxes: 'H' (index 0), 'e' (index 1), 'l' (index 2), 'l' (index 3), and 'o' (index 4). A variable 'x' is shown pointing to the 'o' character at index 4. Below the diagram, a code block shows the loop: `for x in s1:` followed by `print(x)`.

Traversing a String



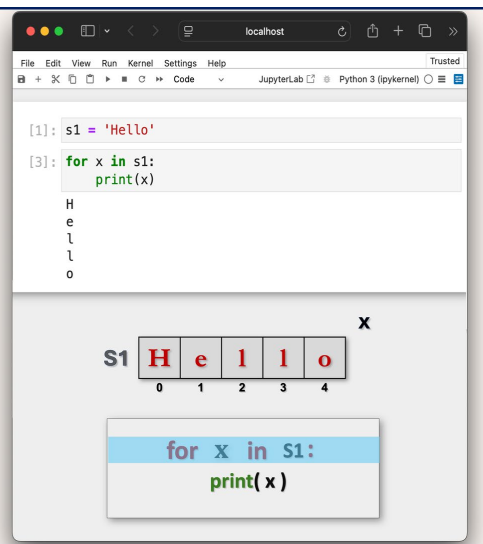
The image shows a JupyterLab interface with a code editor and an output area. The code editor contains the following code:

```
[1]: s1 = 'Hello'
[3]: for x in s1:
      print(x)
```

The output area displays the characters 'H', 'e', 'l', 'l', 'o' on separate lines. Below the output, there is a diagram illustrating the string 'Hello' as a sequence of characters in memory. The string is represented as a sequence of five boxes, each containing a character: 'H', 'e', 'l', 'l', 'o'. The boxes are indexed from 0 to 4. A variable 'x' is shown above the last box, indicating it is the current character being processed. Below the diagram, there is a code block showing the loop structure:

```
for x in s1:
    print(x)
```

Traversing a String



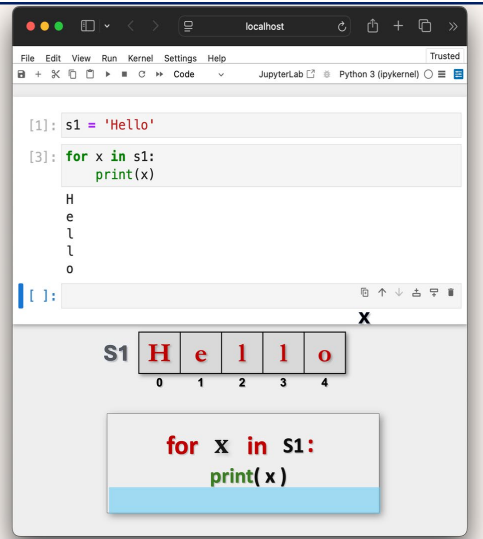
The image shows a JupyterLab interface with a code editor and an output area. The code editor contains the following code:

```
[1]: s1 = 'Hello'
[3]: for x in s1:
      print(x)
```

The output area displays the characters 'H', 'e', 'l', 'l', 'o' on separate lines. Below the output, there is a diagram illustrating the string 'Hello' as a sequence of characters in memory. The string is represented as a sequence of five boxes, each containing a character: 'H', 'e', 'l', 'l', 'o'. The boxes are indexed from 0 to 4. A variable 'x' is shown above the last box, indicating it is the current character being processed. Below the diagram, there is a code block showing the loop structure:

```
for x in s1:
    print(x)
```


Traversing a String

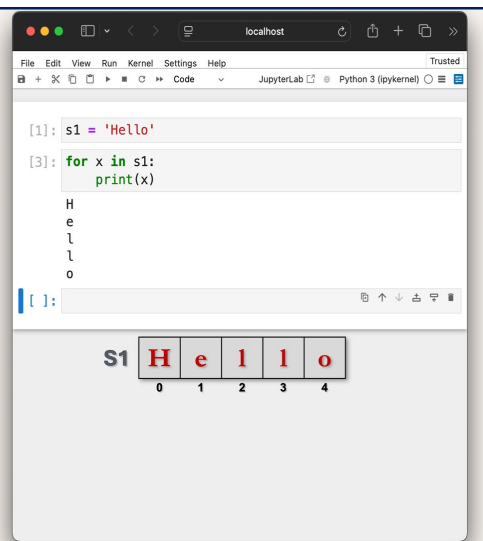


The image shows a JupyterLab interface with a code editor and an output area. The code editor contains the following Python code:

```
[1]: s1 = 'Hello'
[3]: for x in s1:
      print(x)
```

The output area displays the characters 'H', 'e', 'l', 'l', 'o' on separate lines. Below the output, there is a diagram illustrating the string 'Hello' as a sequence of characters in memory, indexed from 0 to 4. The characters are 'H', 'e', 'l', 'l', 'o'. Below the diagram, the code snippet `for x in s1: print(x)` is shown again, with the variable `x` highlighted in blue.

Traversing a String

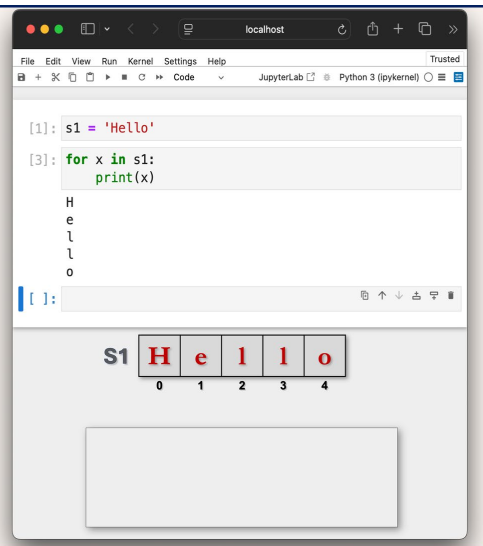


The image shows a JupyterLab interface with a code editor and an output area. The code editor contains the following Python code:

```
[1]: s1 = 'Hello'
[3]: for x in s1:
      print(x)
```

The output area displays the characters 'H', 'e', 'l', 'l', 'o' on separate lines. Below the output, there is a diagram illustrating the string 'Hello' as a sequence of characters in memory, indexed from 0 to 4. The characters are 'H', 'e', 'l', 'l', 'o'. Below the diagram, the code snippet `for x in s1: print(x)` is shown again, with the variable `x` highlighted in blue.

Traversing a String



The image shows a JupyterLab interface on a localhost browser. The code editor contains two cells: the first cell has `s1 = 'Hello'` and the second cell has a `for` loop `for x in s1: print(x)`. The output of the second cell shows the characters 'H', 'e', 'l', 'l', 'o' printed on separate lines. Below the code editor, a variable inspector shows the variable `s1` as a string, with its characters 'H', 'e', 'l', 'l', 'o' displayed in individual boxes with their corresponding indices 0 through 4.

```
[1]: s1 = 'Hello'
```

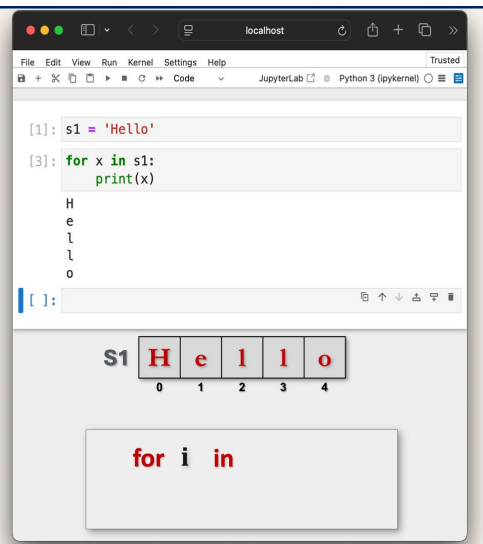
```
[3]: for x in s1:
      print(x)
```

H
e
l
l
o

[]:

s1 H e l l o
0 1 2 3 4

Traversing a String



The image shows a JupyterLab interface on a localhost browser. The code editor contains two cells: the first cell has `s1 = 'Hello'` and the second cell has a `for` loop `for x in s1: print(x)`. The output of the second cell shows the characters 'H', 'e', 'l', 'l', 'o' printed on separate lines. Below the code editor, a variable inspector shows the variable `s1` as a string, with its characters 'H', 'e', 'l', 'l', 'o' displayed in individual boxes with their corresponding indices 0 through 4. A text box below the variable inspector contains the text `for i in`.

```
[1]: s1 = 'Hello'
```

```
[3]: for x in s1:
      print(x)
```

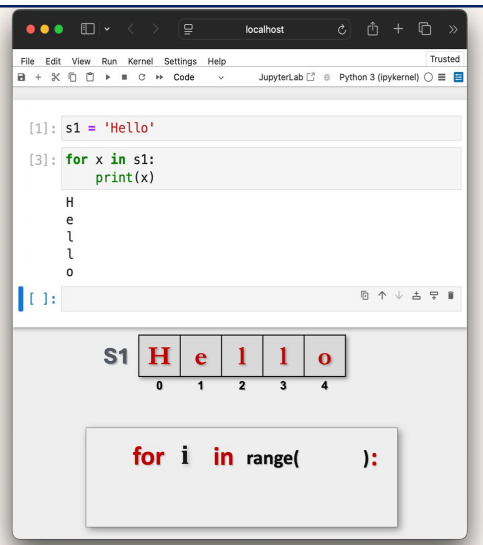
H
e
l
l
o

[]:

s1 H e l l o
0 1 2 3 4

for i in

Traversing a String



The image shows a JupyterLab interface with a code editor and an output area. The code editor contains two cells: the first cell has `s1 = 'Hello'` and the second cell has a `for` loop that iterates over `s1` and prints each character. The output area shows the characters 'H', 'e', 'l', 'l', 'o' printed on separate lines. Below the output, a diagram shows the string `s1` as a sequence of characters in boxes, indexed from 0 to 4. At the bottom, a code snippet shows the start of a `for` loop: `for i in range( ):`.

```
[1]: s1 = 'Hello'
```

```
[3]: for x in s1:  
      print(x)
```

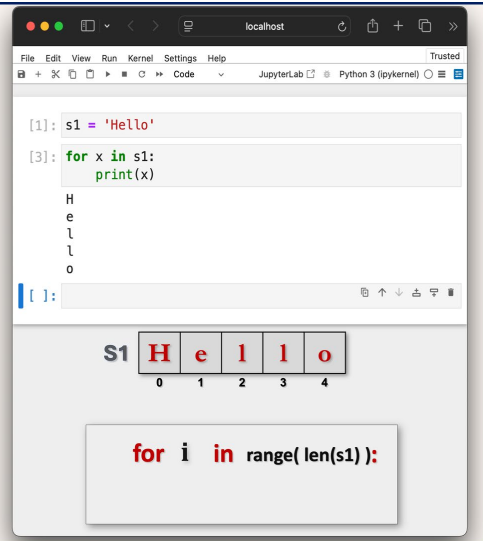
H
e
l
l
o

[]:

s1 H e l l o
0 1 2 3 4

for i in range():

Traversing a String



The image shows a JupyterLab interface with a code editor and an output area. The code editor contains the following Python code:

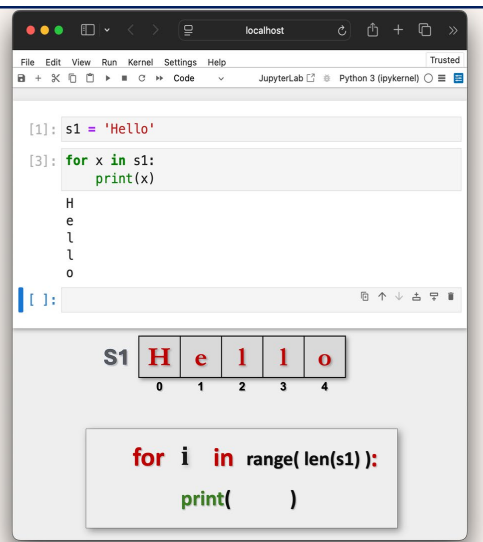
```
[1]: s1 = 'Hello'
[3]: for x in s1:
      print(x)
```

The output area displays the result of the code execution, which is the string 'Hello' printed character by character. Below the output, there is a visual representation of the string 'Hello' as a sequence of characters in a box, with indices 0 through 4 below each character. The characters are 'H', 'e', 'l', 'l', 'o'.

Below the visual representation, there is a code snippet for a for loop that iterates over the range of the length of the string 's1' and prints the character at each index:

```
for i in range(len(s1)):
```

Traversing a String



The image shows a JupyterLab interface with a code editor and an output area. The code editor contains the following Python code:

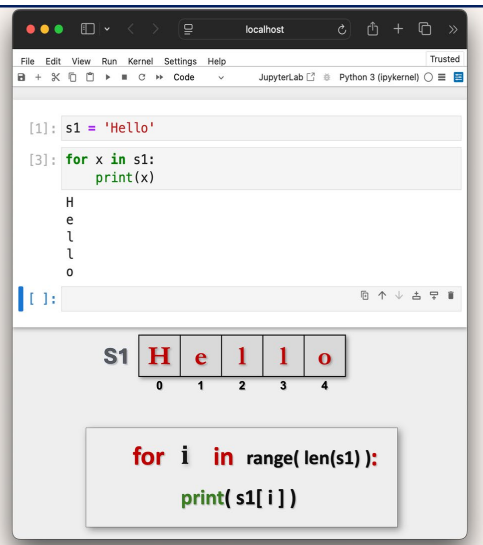
```
[1]: s1 = 'Hello'
[3]: for x in s1:
      print(x)
```

The output area displays the result of the code execution, which is the string 'Hello' printed character by character. Below the output, there is a visual representation of the string 'Hello' as a sequence of characters in a box, with indices 0 through 4 below each character. The characters are 'H', 'e', 'l', 'l', 'o'.

Below the visual representation, there is a code snippet for a for loop that iterates over the range of the length of the string 's1' and prints the character at each index:

```
for i in range(len(s1)):
    print( )
```

Traversing a String



The screenshot shows a JupyterLab window with the following content:

```
[1]: s1 = 'Hello'
```

```
[3]: for x in s1:  
      print(x)
```

The output of the code is:

```
H  
e  
l  
l  
o
```

Below the code editor, there is a visual representation of the string 'Hello' as an array:

s1	H	e	l	l	o
	0	1	2	3	4

At the bottom, a code block shows an alternative way to traverse the string:

```
for i in range(len(s1)):  
    print(s1[i])
```

String Literals

String Literals

String Literals

s1 = 'Hello'

String Literals

S1 = 'Hello'

S2 = "Hello"

String Literals

S1 = 'Hello'

S2 = "Hello"

S3 = '''Hello'''

String Literals

S1 = 'Hello'

S2 = "Hello"

S3 = '''Hello'''

S4 = """Hello"""

String Literals

Single Quotes : **S1 = 'Hello'**

S2 = "Hello"

S3 = '''Hello'''

S4 = """Hello"""

String Literals

Single Quotes : **S1 = 'Hello'**

Double Quotes : **S2 = "Hello"**

S3 = '''Hello'''

S4 = """Hello"""

String Literals

Single Quotes : **S1 = 'Hello'**

Double Quotes : **S2 = "Hello"**

3 Single Quotes : **S3 = '''Hello'''**

S4 = """Hello"""

String Literals

Single Quotes : **S1 = 'Hello'**

Double Quotes : **S2 = "Hello"**

3 Single Quotes : **S3 = '''Hello'''**

3 Double Quotes : **S4 = """Hello"""**

String Literals

Single Quotes : **S1 = 'Hello'**

Double Quotes : **S2 = "Hello"**

3 Single Quotes : **S3 = '''Hello'''**

3 Double Quotes : **S4 = """Hello**

How are you?"""

String Literals

Single Quotes : **S1 = 'Hello'**

Double Quotes : **S2 = "Hello"**

3 Single Quotes : **S3 = '''Hello'''**

3 Double Quotes : **S4 = """Hello**

How are you?"""

Comment

String Literals

Single Quotes : **S1 = 'Hello'**

Double Quotes : **S2 = "Hello"**

3 Single Quotes : **S3 = '''Hello'''**

3 Double Quotes : **S4 = """Hello**

Comment

How are you?"""

S5 = 'Clarks'

String Literals

Single Quotes : **S1 = 'Hello'**

Double Quotes : **S2 = "Hello"**

3 Single Quotes : **S3 = '''Hello'''**

3 Double Quotes : **S4 = """Hello**

How are you?"""

S5 = 'Clark's'

String Literals

Single Quotes : **S1 = 'Hello'**

Double Quotes : **S2 = "Hello"**

3 Single Quotes : **S3 = '''Hello'''**

3 Double Quotes : **S4 = """Hello**

How are you?" """

S5 = "Clark's"

String Literals

Single Quotes : **S1 = 'Hello'**

Double Quotes : **S2 = "Hello"**

3 Single Quotes : **S3 = '''Hello'''**

3 Double Quotes : **S4 = """Hello**

How are you?" """

S5 = Clarks

String Literals

Single Quotes : **S1 = 'Hello'**

Double Quotes : **S2 = "Hello"**

3 Single Quotes : **S3 = '''Hello'''**

3 Double Quotes : **S4 = """Hello**

How are you?"""

S5 = Clark's